

TCP Segment Structure

(Sequence Numbers, Acknowledgments, and Case Study: Telnet)

1. TCP Segment Structure

A TCP segment consists of two main components:

- **Header fields** – contain control and management information.
- **Data field** – carries the actual application data.

When sending large files, TCP divides data into chunks of size limited by the **Maximum Segment Size (MSS)**. For smaller, interactive applications (like Telnet or SSH), data chunks can be as small as a single byte.

Segment Size Example

- TCP header: typically 20 bytes (12 bytes more than UDP).
- Telnet/SSH: often sends 1 byte of data.
- Total segment size $\approx$ 21 bytes.

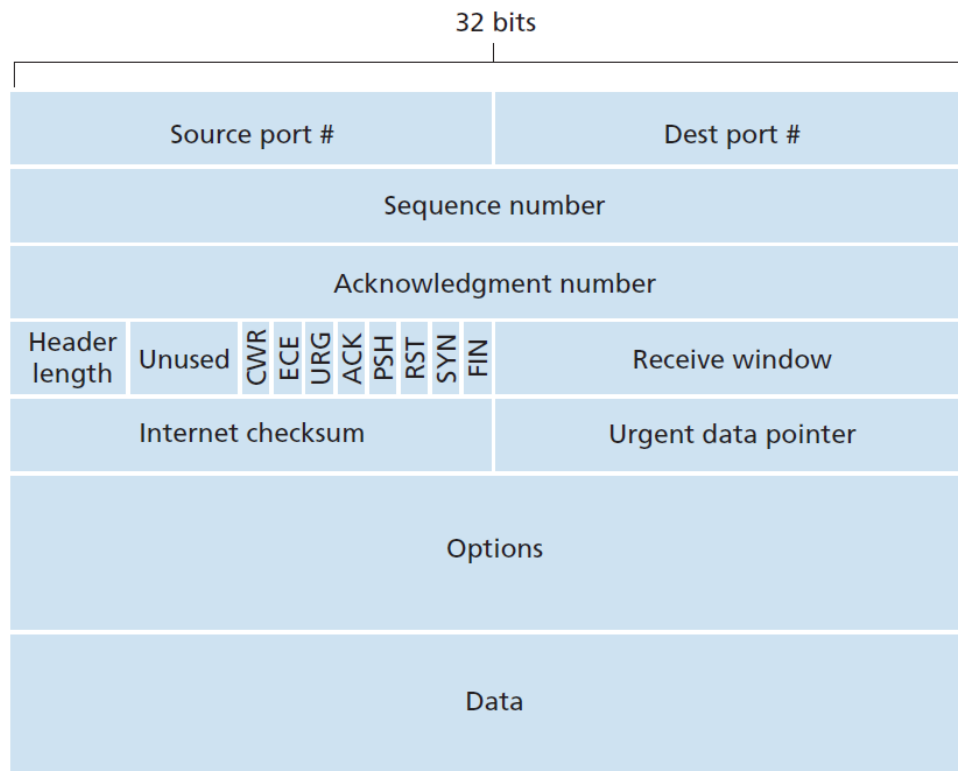


Figure 1: TCP Segment Structure

Key Header Fields

- **Source/Destination Ports** – used for process-level multiplexing.
- **Sequence Number (32 bits)** – first byte in this segment.
- **Acknowledgment Number (32 bits)** – next expected byte.
- **Window Size (16 bits)** – receiver's available buffer size.
- **Header Length (4 bits)** – in 32-bit words.
- **Options** – for MSS, window scaling, timestamps.
- **Checksum** – verifies header + data.
- **Urgent Pointer** – rarely used today.

Flag Bits (6 Control Bits)

ACK	Acknowledgment valid.
SYN	Synchronize sequence numbers.
FIN	Terminate a connection.
RST	Reset a connection.
PSH	Push data to the app immediately.
URG	Indicates urgent data.

Additional Flags

CWR (Congestion Window Reduced) and **ECE** (ECN-Echo) support Explicit Congestion Notification (ECN).

2. Sequence and Acknowledgment Numbers

2.1 Sequence Numbers

- TCP views data as an **ordered stream of bytes**.
- Each segment's **sequence number** represents the byte-stream number of its first byte.

Example

Host A sends a 500,000-byte file to Host B with $MSS = 1,000$ bytes:

- Segment 1 $\rightarrow$ Seq = 0
- Segment 2 $\rightarrow$ Seq = 1000
- Segment 3 $\rightarrow$ Seq = 2000

$\Rightarrow$ Total = 500 segments.

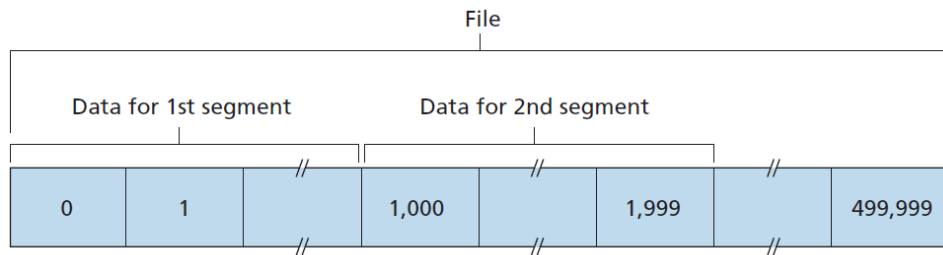


Figure 2: Dividing Data into TCP Segments

2.2 Acknowledgment Numbers

- Indicate the **next byte expected**.
- Use **cumulative ACKs** – all bytes before it have been received.

Example

If Host A received bytes 0–535, $\Rightarrow$ next expected = 536, so ACK = 536.

Initial Sequence Numbers

Both hosts select random initial sequence numbers to avoid confusion with delayed segments.

3. Case Study: Telnet

Telnet (RFC 854) is an interactive TCP-based protocol that sends each typed character to the remote host and echoes it back.

Security Note

Telnet transmits data **in plain text**. Use **SSH (Secure Shell)** for modern secure remote access.

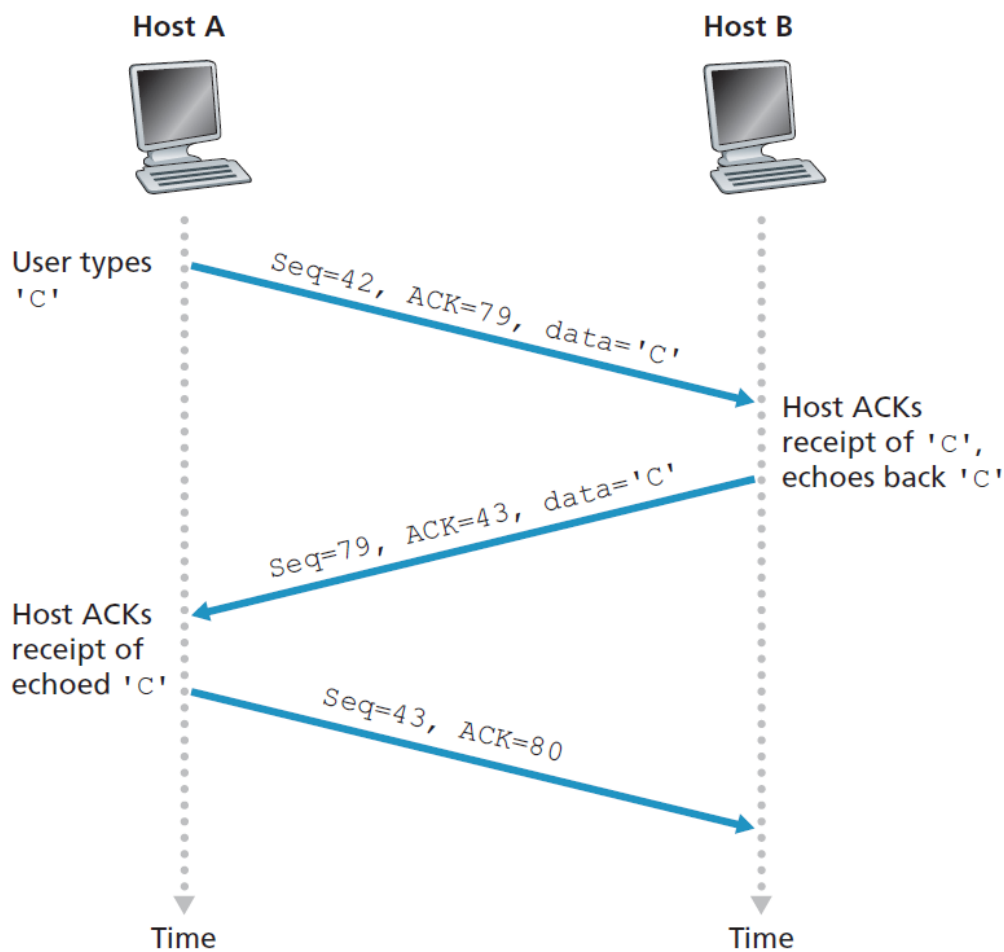


Figure 3: Telnet Example – Sequence and Acknowledgment Flow

Example Flow

1. Client sends 'C' (Seq=42, Ack=79)
2. Server echoes (Seq=79, Ack=43)
3. Client acknowledges (Seq=43, Ack=80)

Concept: Piggybacking

ACKs are often **piggybacked** on data segments — increasing efficiency by avoiding extra empty ACK packets.

4. Summary

TCP Highlights

- TCP segment = Header + Data
- Headers include ports, sequence, and ACK numbers
- Uses cumulative ACKs for reliability
- Telnet demonstrates real-time acknowledgment flow